

Introduction:

Timing attacks occur when variations in execution time reveal information about secret data. In web environments, JavaScript functions can become attack vectors if an adversary can measure how long a function takes to run. This project explores a simple image comparison algorithm implemented in the browser. The experiment investigates whether differences in runtime can reveal information about how similar two images are, and whether this timing information can be used to infer something about a private image. The study also evaluates a constant-time version of the same algorithm to determine whether such a modification can prevent timing-based leakage.

Experimental Design:

Two JavaScript functions were implemented to compare images represented as Uint8Array byte arrays of RGBA pixel data:

1. `naiveCompare`: Iterates over all pixels and exits immediately upon finding a mismatch.
 - Expected to leak timing information, since more similar images take longer to compare.
 - Algorithm code can be found in lines 81-90 of `color_attack.html`
2. `constantCompare`: Iterates through all pixels regardless of differences, using a combined XOR operation to determine equality only after the entire image has been scanned.
 - Expected to have consistent runtime across all inputs of the same size.
 - Algorithm code can be found in lines 92-100 of `color_attack.html`

Each image used in testing measured 128×128 pixels. All experiments were run in the browser using `performance.now()` for precise timing.

Methods:

1. Similarity Experiment:

A random base image was generated and modified to create versions that were 0%, 25%, 50%, 75%, and 100% identical to it. Each comparison was run for hundreds of trials to reduce noise, with warm-up iterations to allow the JavaScript engine to optimize code paths. The mean runtime for each condition was recorded for both algorithms.

2. Color-Guess Attack

A simulated attacker attempted to discover the color of a secret solid-color image by comparing it against 27 candidate solid-color images from a fixed “web-safe” palette. The attacker measured the runtime for each comparison using the naïve algorithm. Because images that shared a longer matching prefix with the secret color produced slightly longer runtimes, the attacker could identify which candidate most likely matched the secret.

Results:

When using naiveCompare, average runtime increased as image similarity increased. The algorithm performed more work before exiting, leading to longer measured execution times. constantCompare produced nearly identical runtimes for all similarity levels, confirming that its behavior was constant with respect to input data.

```
Generating base image 128x128 (RGBA = 65536 bytes) ...
Running experiments: 600 trials per (variant, algorithm) ...
Variant 0% ...
    naive mean=0.0000 ms, constant mean=0.0367 ms
Variant 25% ...
    naive mean=0.0117 ms, constant mean=0.0383 ms
Variant 50% ...
    naive mean=0.0283 ms, constant mean=0.0350 ms
Variant 75% ...
    naive mean=0.0417 ms, constant mean=0.0383 ms
Variant 100% ...
    naive mean=0.0550 ms, constant mean=0.0350 ms
Done. Click "Download CSV" to save results.
```

Results from image comparison experiment, full CSV (image_compare_timings.csv) is included in the project file.

In the color-guess experiment, the attacker successfully identified the secret image’s color by ranking candidates based on their mean runtimes. The correct color consistently appeared among the slowest comparisons.

```
Testing 27 web-safe candidates (trials=300, warmup=150)
tested 1/27...
tested 9/27...
tested 17/27...
tested 25/27...
Done. Guessed: #ff0080 (mean 0.036667 ms)
```

Candidate	Swatch	Mean (ms)	StdDev (ms)	Exact
#ff0080		0.036667	0.187942	X
#00ff00		0.003333	0.057639	
#800080		0.003333	0.057639	
#000000		0.000000		

Results from the color-guess experiment. Full CSV (color_results.csv) is included in the project file.

Discussion:

These results demonstrate that the naïve image comparison algorithm leaks information through execution time. The amount of time the function takes to complete is correlated with how similar the compared images are, making it possible to infer partial information about the private data being compared. In the solid-color attack, the timing differences were strong enough for an attacker to correctly determine the hidden color using only timing measurements.

The constant-time comparison effectively prevents this form of leakage by ensuring the same number of operations are performed regardless of input data. Although constant-time code may slightly reduce performance for dissimilar inputs, the tradeoff is minimal compared to the privacy risk avoided.

A note on jitter:

At one point, random timing jitter was added to the naïve comparison (lines 71-84 in image_comparison.html) to try to hide the timing pattern. However, this did not change the results in any meaningful way. The timing leak was still there, and the attack still worked once the results were averaged over many trials. Because jitter only added noise and did not improve privacy, it was removed from the final version of the project.